Kubectl Apply Command

In this article, we dive into the inner workings of the `kubectl apply` command and explain how Kubernetes manages object configurations declaratively. If you're looking to understand how local configuration files interact with live objects in the Kubernetes cluster, read on.

Local Configuration Example

Consider the following local configuration file (`nginx.yaml`) that defines a simple Nginx Pod:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end-service
spec:
  containers:
    - name: nginx-container
      image: nginx:1.18
```

You can apply this configuration using any of the commands below:

```
kubectl apply -f nginx.yaml
kubectl apply -f /path/to/config-files
kubectl apply -f nginx.yaml
```

How Kubectl Apply Works

When you execute the `kubectl apply` command, Kubernetes considers three sources before making any changes:

1. The local configuration file.
2. The live object's configuration present in the Kubernetes cluster.
3. The last applied configuration stored on the live object as an annotation.

Note

If the object does not exist in the cluster, Kubernetes creates it using the local configuration. The newly created object resembles the file you provided but includes additional fields (such as status information) added by the cluster.

For example, after creating the object, it might look similar to this:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end-service
spec:
```

```
  containers:
    - name: nginx-container
      image: nginx:1.18
status:
  conditions:
    - lastProbeTime: null
      status: "True"
      type: Initialized
```

The Last Applied Configuration

When you run:

```
kubectl apply -f nginx.yaml
```

the YAML file is converted to JSON and stored as the "last applied configuration" in the live object's metadata under an annotation. During subsequent `**apply**` operations, Kubernetes compares the updated local configuration with the live configuration, using the stored annotation to compute any changes.

For instance, if you update the image version from 1.18 to 1.19 in your local file:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end-service
spec:
  containers:
    - name: nginx-container
      image: nginx:1.19
```

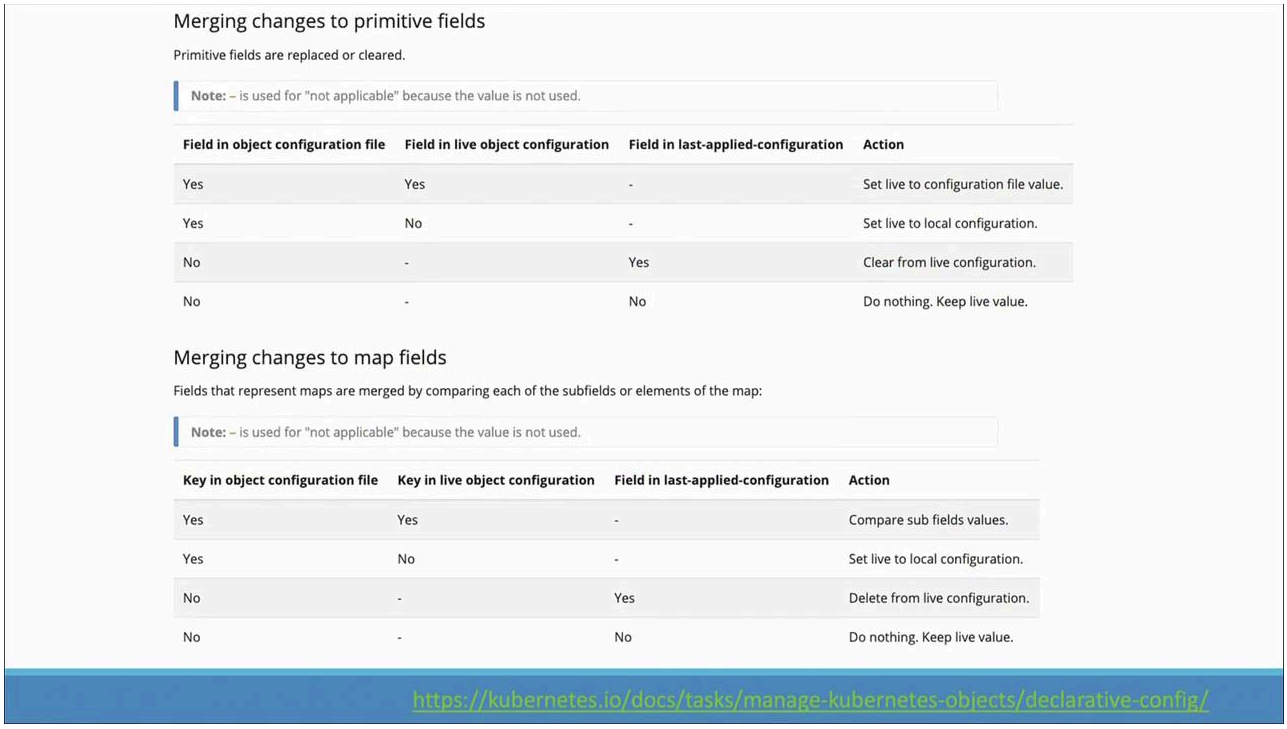
and run:

```
kubectl apply -f nginx.yaml
```

the command compares the new image value with the live configuration. Any differences prompt Kubernetes to update the live object. Moreover, if a field (for example, a label) is removed from your local file, the command will remove it from the live configuration as well, based on the last applied configuration.

Merge Strategy for Changes

The following diagram explains how Kubernetes merges changes for both primitive and map fields. It details the actions taken based on the presence or absence of a field in the local, live, or last applied configuration:



Merging changes to primitive fields

Primitive fields are replaced or cleared.

Note: – is used for "not applicable" because the value is not used.

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

Merging changes to map fields

Fields that represent maps are merged by comparing each of the subfields or elements of the map:

Note: – is used for "not applicable" because the value is not used.

Key in object configuration file	Key in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Compare sub fields values.
Yes	No	-	Set live to local configuration.
No	-	Yes	Delete from live configuration.
No	-	No	Do nothing. Keep live value.

https://kubernetes.io/docs/tasks/manage-kubernetes-objects/declarative-config/

Last Applied Configuration Storage

One crucial aspect of the **`kubectl apply`** process is that the last applied configuration is stored as an annotation within the live object's metadata.

Consider the following updated local configuration file:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
    - name: nginx-container
      image: nginx:1.19
```

When you run:

```
kubectl apply -f nginx.yaml
```

Kubernetes stores the last applied configuration. Here is an example of what the JSON representation of this configuration might look like:

```
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "annotations": {},
    "labels": {
      "run": "myapp-pod"
    },
    "name": "myapp-pod"
  },
  "spec": {
    "containers": [
      {
        "image": "nginx:1.19",
        "name": "nginx-container"
      }
    ]
  }
}
```

If you inspect the live object, you will find an annotation similar to the one below, embedded in the metadata:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: '{"apiVersion":"v1","kind":
  labels:
    app: myapp
    type: front-end-service
spec:
  containers:
    - name: nginx-container
      image: nginx:1.18
status:
  conditions: null
```

Warning

Only the `kubectl apply` command stores the last applied configuration. Commands like `kubectl create` or `kubectl replace` do not maintain this history. To ensure accurate tracking of changes, always use the declarative approach with `kubectl apply`.

Conclusion

This lesson has provided insight into the internal mechanics of the `kubectl apply` command and how Kubernetes manages changes to object configurations. By storing the last applied configuration as an annotation, Kubernetes simplifies the process of comparing and merging configuration changes. Use this knowledge to maintain a robust and declarative workflow in your Kubernetes deployments.

For more information on working with Kubernetes, be sure to explore the following links:

- **Kubernetes Documentation**
- **Kubernetes Basics**

Watch Video

Watch video content

Previous

Imperative vs Declarative

Next

Kubernetes Namespaces